# design

## System Blueprint (***a.k.a.*** "Design Doc")

### TNPG: TACOS

### project: P05 Le Fin

### Target ship date: {2026-05-30}

---

**roster:**

| Name | Email | Primary Role | Secondary Role |
|---|---|---|---|
| Ashley Li | ashleyl503@nycstudents.net | PM | - |
| Naomi Kurian | naomik30@nycstudents.net | Primary VM | - |
| Isabel Zheng | isabelz13@nycstudents.net | Secondary VM | - |
| Veronika Duvanova | veronikad7@nycstudents.net | Tertiary VM | Scribe |

---

## Summary

We are creating a collaborative recipe sharing application where a user can share their recipes as well as find recipes based on the foods they have at home.

### Problem Being Solved

The problem being solved is the difficulty of finding recipes that you're able to follow with the ingredients you have at hand.

### Target Users

Who will use this system?

- Beginner level cooks

- People with limited access to food products
- Intermediate level cooks looking to share their recipes

### Why This Project Matters

This project makes cooking accessible to anyone who wants to experiment with a new recipe or produce something out of the ingredients they have accessible to them with minimal to no food shopping needed.

---

## Minimum Viable Product (MVP) Scope

### Core Features (Required for Final Submission)

Features that **must** be completed: 1. The ability to post/share a recipe 1. The ability to browse all recipes 1. The ability to search up specific recipes based on recipe title / ingredients

### Stretch Features (Only if MVP is Complete)

1. The ability to have a profile and see your shared / saved recipes
2. A game / something entertaining for the user to do
3. Use of API for imagery

### Explicit Non-Goals

Features intentionally excluded: - Direct communication between users

---

## Technology Stack

| Layer | Selected Tool |
| --- | --- |
| Backend Framework | Flask |
| Frontend Framework | Tailwind |
| Database | SQLite |
| Authentication | Flask sessions |
| ORM / DB Library | n/a |

### Why This Stack Was Chosen

We chose flask because it provides smooth transition between app calls, selected tailwind because we have higher familiarity with the framework, selected SQLite because it serves our purpose well (recipes don't require complex database).

---

## Team Ownership Plan

Each member must own meaningful deliverables.

| Team Member | Primary Ownership | Secondary Ownership | Specific Deliverables |
|---|---|---|---|
| Ashley | Frontend | Visual Design | search bar |
| Naomi | Backend | Styling | Login/register, recipe html pages |
| Isabel | Backend | Example accounts | profile page, browsing page |
| Veronika | Backend | Visual Design | home html, game html |

---

## Component map

```
---
config:
  layout: elk
---
graph TB
    classDef backendStyle stroke:#818cf8,fill:#eef2ff,color:#1e1b4b
    classDef frontendStyle stroke:#2dd4bf,fill:#f0fdfa,color:#1e1b4b
    classDef dbStyle stroke:#f87171,fill:#fef2f2,color:#1e1b4b
    classDef assetStyle stroke:#a78bfa,fill:#f5f3ff,color:#1e1b4b
    classDef routeStyle stroke:#fb923c,fill:#fff7ed,color:#1e1b4b

    subgraph Backend["Backend"]
        Init["__init__.py"]
        DataDB[("data.db")]
        Logout["/logout route"]
    end

    subgraph Frontend["Frontend"]
        Home["HOME<br/>(home.html)"]
        Login["LOGIN<br/>(login.html)"]
        Register["REGISTER<br/>(register.html)"]
        Profile["PROFILE<br/>(profile.html)"]
        Browse["BROWSE<br/>(browse.html)"]
        Recipe["RECIPE<br/>(recipe.html)"]
```

```
        RecipeEdit["EDIT<br/>(recipeEdit.html)"]
        Game["GAME<br/>(game.html)"]
    end

    subgraph Assets["Shared Assets"]
        CSS["style.css"]
        JS["script.js"]
    end

    %% backend relationships
    Init -->|reads/writes| DataDB
    Init -->|"serves all HTML templates"| Frontend
    Init -->|defines route| Logout

    %% logout route
    Logout -->|redirects to| Home

    %% navbar (home connects bidirectionally except login/register)
    Home <--> |navbar| Profile
    Home <--> |navbar| Browse
    Home <--> |navbar| Recipe
    Home <--> |navbar| RecipeEdit
    Home <--> |navbar| Game
    Login <--> |navbar| Register
    Login -->|to| Home
    Register -->|to| Home

    %% shared assets (single line to group)
    CSS -->|"used by all HTML pages"| Frontend
    JS -->|"used by all HTML pages"| Frontend

    class Init backendStyle
    class Logout routeStyle
    class DataDB dbStyle
    class CSS,JS assetStyle
    class Home,Login,Register,Profile,Browse,Recipe,RecipeEdit,Game frontendStyle
```

## Site map

```
---
config:
  layout: elk
---
graph TB
    classDef pageStyle stroke:#2dd4bf,fill:#f0fdfa,color:#1e1b4b
```

```
    classDef homeStyle stroke:#818cf8,fill:#eef2ff,color:#1e1b4b

    Home["HOME<br/>(home.html)"]:::homeStyle
    Login["LOGIN<br/>(login.html)"]:::pageStyle
    Register["REGISTER<br/>(register.html)"]:::pageStyle
    Profile["PROFILE<br/>(profile.html)"]:::pageStyle
    Browse["BROWSE<br/>(browse.html)"]:::pageStyle
    Recipe["RECIPE<br/>(recipe.html)"]:::pageStyle
    RecipeEdit["EDIT<br/>(recipeEdit.html)"]:::pageStyle
    Game["GAME<br/>(game.html)"]:::pageStyle

    %% connections
    Home <--> |navbar| Profile
    Home <--> |navbar| Browse
    Home <--> |navbar| Recipe
    Home <--> |navbar| RecipeEdit
    Home <--> |navbar| Game

    Login <--> |navbar| Register
    Login -->|user login| Home
    Register -->|to| Home
```

### Key User Stories

#### eg0

As a low-income mom, I want to find recipes with food I already own so that I'm not wasting it and letting it go bad.

#### eg1

As a intermediate chef, I want to share recipes so that people of any cooking level can follow along.

#### eg2

As a student, I want to find quick recipes so that I can make a quick meal to eat after class.

## Database Design

```
---
config:
  layout: elk
---
erDiagram
```

```
USERS {
    INTEGER userid PK
    TEXT username
    TEXT password
    INTEGER contributions
    TEXT ingredients
    TEXT favorites
}

RECIPES {
    INTEGER id PK
    INTEGER author FK
    TEXT name
    TEXT description
    TEXT ingredients
    TEXT pic
    TEXT difficulty
}
```

## Testing Plan

1. Work with a single VM at the beginning, we want to be able to sync user profiles with recipes. Each user should be set up with a password, username, and potentially status (active, not online, or completely offline).
2. After databases are set up, information saved, testing the blog-like recipe inputs the user makes & ensuring these are stored properly.

## Timeline

### Week 1 Goals: Set up users page (use it as the foundation) and organize databases.

### Week 2 Goals: Implement the user recipe inputs setting, making it viewable to everyone.

### Week 3 Goals: Use an API to suggest recipes with the user-provided ingredients and accessibility.

### Internal Deadlines: Always make sure that there's something running without bugs.

{List milestones your team has identified, in the order they must be completed. Set a target completion date for each.}

# Completion Criteria (*a.k.a.* “Definition of ‘Done’ ”)

Project is considered complete when all of the following are true: 1. User can publish recipes 1. User can browse through recipes (with search tool) 1. User can list ingredients and find matching recipes

# Open Questions

What is the meaning of life if not eating?

# Appendix

N/A

# Other

N/A